

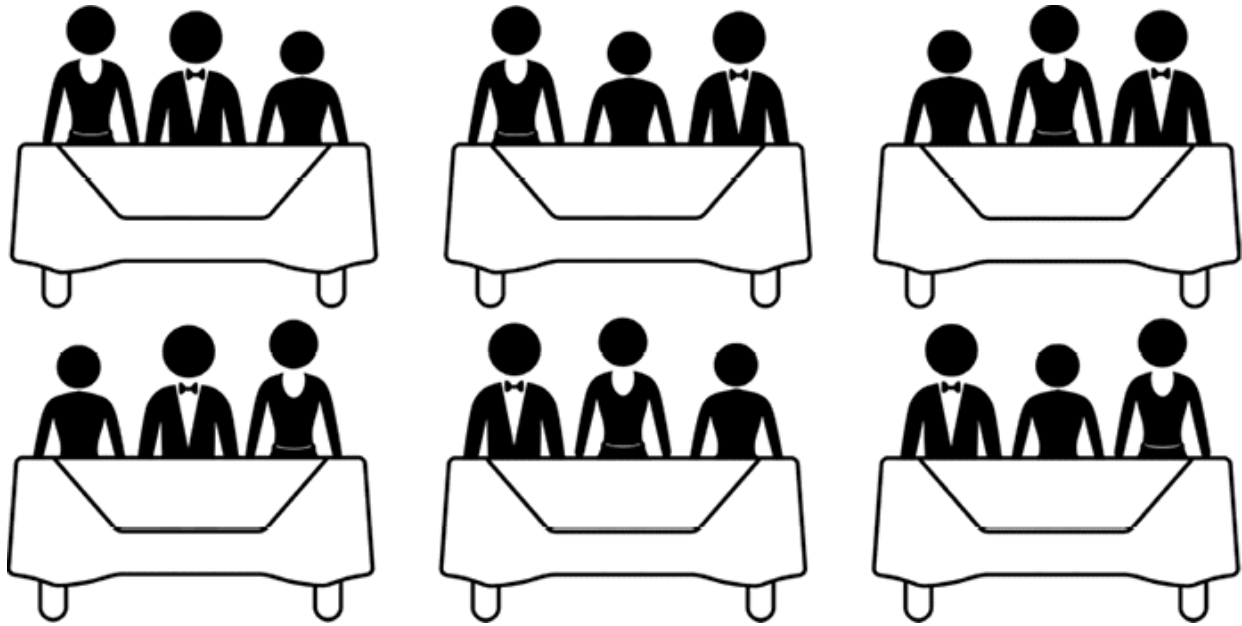


Figure 6-2: All six possible permutations of three wedding guests at a table

Of course, to get every permutation of $\{B, C\}$, we'd recursively repeat the process with B as the head and C as the tail. The permutation of a single character is the character itself; this is our base case. By putting the head B in every possible location in C, we get the BC and CB permutations we used in the previous paragraph. Remember that, while order doesn't matter with sets (as $\{B, C\}$ is the same as $\{C, B\}$), it does matter with permutations (BC is not a duplicate of CB).

Our recursive permutation function accepts as an argument a string of characters and returns an array of strings of every possible permutation of those characters. Let's ask the three questions about our recursive algorithms for this function:

What is the base case? An argument of a single character string or empty string, which returns an array of just that string.

What argument is passed to the recursive function call? The string argument missing one character. A separate recursive call is made for each character missing.

How does this argument become closer to the base case? The size of the string shrinks and eventually becomes a single-character string.

The recursive permutations algorithm is implemented in *permutations.py*: